

Can you provide more examples of complex JOIN queries?

Below are more complex JOIN query patterns you'll see in real projects. I'll use an e-commerce style schema in examples:

- `customers(customer_id, email, city, created_at)`
- `orders(order_id, customer_id, order_date, status, total_amount)`
- `order_items(order_item_id, order_id, product_id, quantity, unit_price)`
- `products(product_id, product_name, category_id, price)`
- `categories(category_id, category_name)`

(You can rename tables/columns to match yours.)

1) Multi-table JOIN + aggregation (revenue by category)

sql

```
1  SELECT
2      c.category_id,
3      c.category_name,
4      SUM(oi.quantity * oi.unit_price) AS revenue
5  FROM orders o
6  JOIN order_items oi ON oi.order_id = o.order_id
7  JOIN products p ON p.product_id = oi.product_id
8  JOIN categories c ON c.category_id = p.category_id
9  WHERE o.status IN ('PAID', 'SHIPPED')
10 GROUP BY c.category_id, c.category_name
11 ORDER BY revenue DESC;
```

2) LEFT JOIN "include zeros" (customer lifetime value even if no orders)

sql

```

1  SELECT
2      cu.customer_id,
3      cu.email,
4      COUNT(DISTINCT o.order_id) AS total_orders,
5      COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS
    lifetime_value
6  FROM customers cu
7  LEFT JOIN orders o
8      ON o.customer_id = cu.customer_id
9      AND o.status <> 'CANCELLED'
10 LEFT JOIN order_items oi
11     ON oi.order_id = o.order_id
12 GROUP BY cu.customer_id, cu.email
13 ORDER BY lifetime_value DESC;

```

Key idea: put `o.status <> 'CANCELLED'` inside the JOIN so customers with only cancelled orders don't disappear.

3) Top N per group (top 3 products per category) — MySQL 8+

Uses window functions (`DENSE_RANK()` / `ROW_NUMBER()`). (dev.mysql.com)

sql

```

1  WITH product_rev AS (
2      SELECT
3          c.category_id,
4          c.category_name,
5          p.product_id,
6          p.product_name,
7          SUM(oi.quantity * oi.unit_price) AS revenue
8      FROM orders o
9      JOIN order_items oi ON oi.order_id = o.order_id
10     JOIN products p ON p.product_id = oi.product_id
11     JOIN categories c ON c.category_id = p.category_id
12     WHERE o.status IN ('PAID', 'SHIPPED')
13     GROUP BY c.category_id, c.category_name, p.product_id,
14             p.product_name
15 ),
16 ranked AS (
17     SELECT
18         *,
19         DENSE_RANK() OVER (PARTITION BY category_id ORDER BY
20             revenue DESC) AS rnk
21     FROM product_rev
22 )
23 SELECT *

```

```
22 FROM ranked
23 WHERE rnk <= 3
24 ORDER BY category_name, rnk, revenue DESC;
```

4) "Latest row per group" (latest order per customer) — MySQL 8+

sql

```
1 WITH ranked_orders AS (
2     SELECT
3         o.*,
4         ROW_NUMBER() OVER (
5             PARTITION BY o.customer_id
6             ORDER BY o.order_date DESC, o.order_id DESC
7         ) AS rn
8     FROM orders o
9     WHERE o.status <> 'CANCELLED'
10 )
11 SELECT
12     ro.customer_id,
13     ro.order_id,
14     ro.order_date,
15     SUM(oi.quantity * oi.unit_price) AS order_value
16 FROM ranked_orders ro
17 JOIN order_items oi ON oi.order_id = ro.order_id
18 WHERE ro.rn = 1
19 GROUP BY ro.customer_id, ro.order_id, ro.order_date;
```

Window functions are supported in MySQL 8.0+. (dev.mysql.com)

5) Anti-join (customers with **no** orders)

sql

```
1 SELECT c.customer_id, c.email
2 FROM customers c
3 LEFT JOIN orders o ON o.customer_id = c.customer_id
4 WHERE o.order_id IS NULL;
```

6) Anti-join (products never sold)

sql

```
1 | SELECT p.product_id, p.product_name
2 | FROM products p
3 | LEFT JOIN order_items oi ON oi.product_id = p.product_id
4 | WHERE oi.product_id IS NULL;
```

(Alternative: WHERE NOT EXISTS (...))

7) Non-equi JOIN (range join: salary band / price slab / tax slab)

Assume salary_bands(band_id, label, min_salary, max_salary):

sql

```
1 | SELECT
2 |     e.emp_id,
3 |     e.salary,
4 |     b.label
5 | FROM employees e
6 | JOIN salary_bands b
7 |     ON e.salary >= b.min_salary
8 |     AND e.salary < b.max_salary;
```

8) Effective-dated JOIN (historical status at order time)

Assume employee_status(emp_id, status, effective_from, effective_to) and orders(salesperson_id, order_date, ...):

sql

```
1 | SELECT
2 |     o.order_id,
3 |     o.order_date,
4 |     s.status
5 | FROM orders o
6 | JOIN employee_status s
7 |     ON s.emp_id = o.salesperson_id
```

```
8 | AND o.order_date >= s.effective_from
9 | AND o.order_date < COALESCE(s.effective_to, '9999-12-31');
```

9) Self-join (employee → manager → director)

Assume employees(emp_id, name, manager_id):

sql

```
1 | SELECT
2 |     e.name AS employee,
3 |     m.name AS manager,
4 |     d.name AS director
5 | FROM employees e
6 | LEFT JOIN employees m ON m.emp_id = e.manager_id
7 | LEFT JOIN employees d ON d.emp_id = m.manager_id;
```

10) Many-to-many JOIN + HAVING (students who took **both** SQL and Java)

Assume students, courses, enrollments(student_id, course_id):

sql

```
1 | SELECT s.student_id, s.name
2 | FROM students s
3 | JOIN enrollments e ON e.student_id = s.student_id
4 | JOIN courses c ON c.course_id = e.course_id
5 | WHERE c.course_name IN ('SQL', 'Java')
6 | GROUP BY s.student_id, s.name
7 | HAVING COUNT(DISTINCT c.course_name) = 2;
```

11) FULL OUTER JOIN (MySQL workaround using UNION)

MySQL doesn't support FULL OUTER JOIN directly; use LEFT JOIN ... UNION ... RIGHT JOIN. (mysqltutorial.org)

sql

```
1 SELECT a.id, a.val, b.val AS b_val
2 FROM table_a a
3 LEFT JOIN table_b b ON b.id = a.id
4
5 UNION
6
7 SELECT a.id, a.val, b.val AS b_val
8 FROM table_a a
9 RIGHT JOIN table_b b ON b.id = a.id;
```

12) UPDATE with JOIN (recompute order totals from order_items)

MySQL supports joins in multi-table UPDATE/DELETE statements. (dev.mysql.com)

sql

```
1 UPDATE orders o
2 JOIN (
3     SELECT order_id, SUM(quantity * unit_price) AS calc_total
4     FROM order_items
5     GROUP BY order_id
6 ) x ON x.order_id = o.order_id
7 SET o.total_amount = x.calc_total
8 WHERE o.status <> 'CANCELLED';
```

13) DELETE with JOIN (delete orphan rows safely)